Artifacts

Ethical Considerations

In the following, we first identify stakeholders and then review ethical considerations for this work according to the principles in the Menlo report. This work has the following three groups of stakeholders: (1) Primary stakeholders (*end-users* of messaging services, *research team*) which are directly affected by this work and future developments which arise from it; (2) Commercial stakeholders (*companies*) which may implement the protocols; (3) Third parties (*regulators, infrastructure providers, society*) who do not actively participate in the implementation or use of the proposed methods but may be affected by it. Now we go over the principles from the Menlo report and detail the potential positive and negative impacts on stakeholders.

Beneficence. *End-users.* This work proposes provably-secure, improved messaging protocols for anonymous key exchange. The security improvements primarily benefit users who want or require privacy in messaging, including those facing heightened risks from surveillance or profiling. In addition, the optimizations proposed for the existing PQXDH protocol reduce bandwidth and computational requirements, thereby lowering resource consumption on end-user devices. While this work does not introduce fundamentally new capabilities for abuse beyond those already present in existing communication systems, anonymous key exchange can be abused for fraudulent or harmful purposes as described in the following. **Harms:** With a sender-anonymous key exchange, a user Alex could disguise their identity or pose as someone else to contact another user Blake who does not want to be contacted by Alex. Establishing such a communication channel without Blake's consent might expose Blake to unwanted or harmful content. **Mitigations:** Messaging services that deploy sender-anonymous key exchange mechanisms typically provide reporting and anti-spam features and actively work on abuse prevention mechanisms. In particular, Signal Messenger, which may directly benefit from the protocol improvements in this work, implements access tokens for spam prevention and uses sender certificates to prevent spoofing.
Companies and infrastructure providers. Messaging providers, including Signal Messenger, that aim to provide anonymous communication may implement our protocols which may require non-trivial engineering effort. The approach presented in this paper is used to reduce redundancy in existing protocols and enable the design of efficient and resource-saving protocols. Companies using this work to improve their existing implementations may benefit from this work as it results in reduced bandwidth and lower computational load on their servers. Conversely, incorporating sender anonymity in key exchange into systems that previously did not support it may increase resource requirements for infrastructure and end-user devices. These considerations similarly affect infrastructure providers for carrying or relaying traffic. Furthermore, companies implementing mechanisms for anonymous communication may need to put additional effort into developing abuse prevention solutions. Any entity implementing (parts of) this work should be aware of this and assess these trade-offs on an individual basis.

Respect for Persons. *End-users, companies, and society.* For this work, we did not collect or analyze user data, nor did we interfere with any user or company operations. We used the free and open source code from Signal Messenger to evaluate protocol performance.
Research team. For their work, the research team studied existing literature in cryptography and used free and open source code for the performance evaluation. The research team did not conduct physical experiments, did not interact with any users (e.g., of messaging services), did not collect or analyze any user data, and was at no point exposed to any harmful content. It was the free will of each team member to participate in and contribute to this work.

Justice. *End-users, companies, and society.* This research does not put any constraints on the usability or availability of existing messaging services or any group who wishes to use our results. Messaging providers can implement the protocols or use the proposed methods at will and there are no special requirements for end-user systems.

Respect for Law and Public Interest. *Regulators and society.* Anonymous communication technologies raise complex legal and policy questions. We present our analysis transparently, cite and refer to related work diligently to support informed public discussion on privacy-preserving communication systems and their societal implications.

Decision to Publish. This work proposes a framework for analyzing, improving, and adding sender-anonymity in key exchange protocols as well as concrete protocol descriptions. We acknowledge that anonymous key exchange may enable unsolicited connection establishment, fraudulent spam and spoofing. Yet, these risks are already accounted for and mitigated by existing prevention mechanisms, e.g., on the application level. We therefore believe that making this research public allows for the development of secure and anonymous systems for people which rely on private communication, promotes informed decision making of actively or passively involved stakeholders, and can facilitate further advances in the cryptographic community.

As detailed above, our internal review did not reveal any ethical risks which outweigh the benefits of publication. We carefully proofread and verified technical claims and followed present research standards. All the above considerations thus lead us to the decision to publish.

Appendix E. Standard Definitions

We introduce definitions for standard cryptographic primitives we use throughout the paper.

E.1. Secret Key Encryption

Syntax. A secret key encryption scheme SKE consists of two algorithms, $\mathrm{SKE.enc}$ and $\mathrm{SKE.dec}$ which each take a

key from keyspace $\mathcal{K}$ in part as input, with the following syntax:

- $\text{SKE.enc} : \mathcal{K} \times \mathcal{M} \to_\$ \mathcal{C}$
- $\text{SKE.dec} : \mathcal{K} \times \mathcal{C} \to \mathcal{M}$

We say that a SKE scheme SKE is *correct* if for all $k \in \mathcal{K}$, $m \in \mathcal{M}$, it holds that $\text{SKE.dec}(k, \text{SKE.enc}(k, m)) = m$.

Game $\text{IND}^b_{\text{SKE}}(\mathcal{A})$
00 $n \leftarrow 0$; $CH \leftarrow \emptyset$; $X \leftarrow \emptyset$
01 $b' \leftarrow\$ \mathcal{A}$
02 Require $CH \cap X = \emptyset$
03 Stop with b'

Oracle Gen
04 $n \leftarrow n + 1$; $CK_n \leftarrow \emptyset$
05 $k_n \leftarrow\$ \mathcal{K}$
06 Return

Oracle $\text{Exp}(i)$
07 $X \xleftarrow{\cup} \{i\}$
08 Return k_i

Oracle $\text{Enc}(i, ch, m)$
09 Require $i \in [n]$
10 $c_0 \leftarrow \text{SKE.enc}(k, m)$
11 If ch:
12 $\quad c_1 \leftarrow\$ \{c \in \mathcal{C} : |c| = |c_1|\}$
13 $\quad CK_i \xleftarrow{\cup} \{c_b\}$
14 $\quad CH \xleftarrow{\cup} \{i\}$
15 $\quad$ Return c_b
16 Return c_0

Figure 17. Multi-instance and exposable indistinguishability game IND for SKE scheme SKE.

Security. We require that a secret key encryption scheme hides all information about the plaintext in a multi-instance setting where keys may be exposed. The adversary must not be able to distinguish whether challenge ciphertexts are real encryptions or random strings of the same length as long as the corresponding secret key is not exposed.

Definition 6. *Consider the game* IND *in Figure 17. We define the advantage of an adversary $\mathcal{A}$ against the multi-instance and exposable indistinguishability of a SKE scheme* SKE *as*

$$\text{Adv}^{\text{ind}}_{\text{SKE}}(\mathcal{A}) \coloneqq \left|\Pr[\text{IND}^0_{\text{SKE}}(\mathcal{A}) = 1] - \Pr[\text{IND}^1_{\text{SKE}}(\mathcal{A}) = 1]\right|.$$

E.2. Signature Scheme

Syntax. A signature scheme S consists of three algorithms, S.gen, S.sgn and S.vfy with the following syntax:

- $\text{S.gen} : \emptyset \to_\$ \mathcal{SK} \times \mathcal{VK}$
- $\text{S.sgn} : \mathcal{SK} \times \mathcal{M} \times \mathcal{R} \to \mathcal{S}$ resp. $\text{S.sgn} : \mathcal{SK} \times \mathcal{M} \to_\$ \mathcal{S}$
- $\text{S.vfy} : \mathcal{VK} \times \mathcal{M} \times \mathcal{S} \to \{0, 1\}$

Game $\text{CORR}_{\text{S}}(\mathcal{A})$
00 $n \leftarrow 0$
01 Invoke $\mathcal{A}$
02 Stop with 0

Oracle Gen
03 $n \leftarrow n + 1$
04 $(ssk_n, svk_n) \leftarrow\$ \text{S.gen}$
05 Return (svk_n, ssk_n)

Oracle $\text{Sgn}(i, m)$
06 Require $i \in [n]$
07 $\sigma \leftarrow\$ \text{S.sgn}(ssk_i, m)$
08 If $\text{S.vfy}(spk_i, m, \sigma) = \texttt{fal}$:
09 $\quad$ Stop with 1
10 Return σ

Figure 18. Multi-instance correctness game for signature scheme S.

Definition 7. *Consider the game* CORR *in Figure 18. We define the correctness error of signature scheme* S *as*

$$\text{Adv}^{\text{corr}}_{\text{S}}(\mathcal{A}) \coloneqq \Pr[\text{CORR}_{\text{S}}(\mathcal{A}) \Rightarrow 1].$$

Game $\text{SUF}_{\text{S}}(\mathcal{A})$
00 $n \leftarrow 0$; $XP \leftarrow \emptyset$
01 Invoke $\mathcal{A}$
02 Stop with 0

Oracle Gen
03 $n \leftarrow n + 1$
04 $(ssk_n, svk_n) \leftarrow\$ \text{S.gen}$
05 $MS_n[\cdot] \leftarrow \emptyset$
06 Return svk_n

Oracle $\text{Exp}(i)$
07 Require $i \in [n]$
08 $XP \xleftarrow{\cup} \{i\}$
09 Return ssk_i

Oracle $\text{Sgn}(i, m)$
10 Require $i \in [n]$
11 $\sigma \leftarrow\$ \text{S.sgn}(ssk_i, m)$
12 $MS_i[m] \xleftarrow{\cup} \{\sigma\}$
13 Return σ

Oracle $\text{Vfy}(i, m, \sigma)$
14 Require $i \in [n]$
15 $v \leftarrow \text{S.vfy}(svk_i, m, \sigma)$
16 If $i \notin XP \wedge \sigma \notin MS_i[m]$:
17 $\quad$ If $v = 1$: Stop with 1
18 Return v

Figure 19. Multi-instance and exposable strong unforgeability security game SUF for signature scheme S.

Security.

Definition 8. *Consider the game* SUF *in Figure 19. We define the advantage of an adversary $\mathcal{A}$ against the multi-instance and exposable one-time strong unforgeability of a signature scheme* S *as*

$$\text{Adv}^{\text{suf}}_{\text{S}}(\mathcal{A}) \coloneqq \Pr[\text{SUF}_{\text{S}}(\mathcal{A}) \Rightarrow 1].$$

E.3. Ring Signature Scheme

Syntax. A ring signature scheme RS consists of three algorithms, RS.gen, RS.sgn and RS.vfy with the following syntax:

- $\text{RS.gen} : \emptyset \to_\$ \mathcal{SK} \times \mathcal{VK}$
- $\text{RS.sgn} : \mathcal{VK}^+ \times \mathcal{SK} \times \mathcal{M} \times \mathcal{R} \to \mathcal{S}$ resp. $\text{S.sgn} : \mathcal{VK}^+ \times \mathcal{SK} \times \mathcal{M} \to_\$ \mathcal{S}$[5]
- $\text{RS.vfy} : \mathcal{VK}^+ \times \mathcal{M} \times \mathcal{S} \to \{0, 1\}$

Game $\text{CORR}_{\text{RS}}(\mathcal{A})$
00 $n \leftarrow 0$
01 Invoke $\mathcal{A}$
02 Stop with 0

Oracle Gen
03 $n \leftarrow n + 1$
04 $(rsk_n, rvk_n) \leftarrow\$ \text{RS.gen}$
05 Return (svk_n, ssk_n)

Oracle $\text{Sgn}(i, m, RVK)$
06 Require $i \in [n]$
07 $RVK \xleftarrow{\cup} \{rvk_i\}$
08 $\sigma \leftarrow\$ \text{RS.sgn}(RVK, rsk_i, m)$
09 If $\text{RS.vfy}(RVK, m, \sigma) = \texttt{fal}$:
10 $\quad$ Stop with 1
11 Return σ

Figure 20. Multi-instance correctness game for ring signature scheme RS.

Definition 9. *Consider the game* CORR *in Figure 20. We define the correctness error of signature scheme* RS *as*

$$\text{Adv}^{\text{corr}}_{\text{RS}}(\mathcal{A}) \coloneqq \Pr[\text{CORR}_{\text{RS}}(\mathcal{A}) \Rightarrow 1].$$

Security. We define unforgeability security for a ring signature scheme RS. Note that we do not define anonymity as we do not analyze deniability in this work.

Definition 10. *Consider the game* UNF *in Figure 21. We define the advantage of an adversary $\mathcal{A}$ against the multi-instance and exposable unforgeability of a ring signature scheme* RS *as*

$$\text{Adv}^{\text{unf}}_{\text{RS}}(\mathcal{A}) \coloneqq \Pr[\text{UNF}_{\text{RS}}(\mathcal{A}) \Rightarrow 1].$$

5. The notation S^+ here for set S denotes "one or more element of S" à la regular expressions.

Game $\mathrm{UNF}_{\mathrm{RS}}(\mathcal{A})$
00 $n \leftarrow 0$; $XVK \leftarrow \emptyset$
01 Invoke $\mathcal{A}$
02 Stop with 0

Oracle Gen
03 $n \leftarrow n + 1$
04 $(rsk_n, rvk_n) \leftarrow\$ \mathrm{RS.gen}$
05 $MS_n[\cdot] \leftarrow \emptyset$
06 Return rvk_n

Oracle $\mathrm{Exp}(i)$
07 Require $i \in [n]$
08 $XVK \xleftarrow{\cup} \{rvk_i\}$
09 Return rsk_i

Oracle $\mathrm{Sgn}(RVK, i, m)$
10 Require $i \in [n]$
11 Require $rvk_i \in RVK$
12 $\sigma \leftarrow\$ \mathrm{RS.sgn}(RVK, rsk_i, m)$
13 $MS_i[m] \xleftarrow{\cup} \{RVK\}$
14 Return σ

Oracle $\mathrm{Vfy}(RVK, m, \sigma)$
15 Require $i \in [n]$
16 $v \leftarrow \mathrm{RS.vfy}(RVK, m, \sigma)$
17 If $RVK \not\subseteq XVK \wedge RVK \notin MS_i[m]$:
18 If $v = 1$: Stop with 1
19 Return v

Figure 21. Multi-instance and exposable unforgeability security game UNF for ring signature scheme RS.

E.4. Diffie-Hellman Key Exchange

Let (G, g, q) be a group of order q with generator g. Two parties, Alex and Bob independently sample private exponents $\mathrm{EK}_A^{\mathrm{sk}}, \mathrm{EK}_B^{\mathrm{sk}} \in \mathbb{Z}_q$ and compute the respective public key $A = g^{\mathrm{EK}_A^{\mathrm{sk}}}, B = g^{\mathrm{EK}_A^{\mathrm{sk}}}$. They exchange the public values and each derive the shared secret $\mathrm{ss} \leftarrow \mathrm{DH}(\mathrm{EK}_A^{\mathrm{sk}}, B) = B^{\mathrm{EK}_A^{\mathrm{sk}}} = A^{\mathrm{EK}_B^{\mathrm{sk}}} = \mathrm{DH}(\mathrm{EK}_B^{\mathrm{sk}}, A)$. For readability, we sometimes change the order of secret and public key in DH when it is clear which key is the secret key.

E.5. Gap Diffie-Hellman

Security. We introduce a multi-instance Gap Diffie-Hellman (GDH) security notion with corruptions. This assumption loses a square root factor to single-instance GDH in existing reductions in the literature due to the use of a forking lemma [51], although in a generic group model (GGM) we do not; see [21], [52] for further details.

Definition 11. *Consider the game* GDH *in Figure 22. Let* (G, g, q) *be a group of order* q *with generator* g*. We define the advantage of an adversary* $\mathcal{A}$ *against the multi-instance and exposable Gap Diffie-Hellman security of DH scheme* DH *as*

$$\mathrm{Adv}^{\mathrm{gdh}}_{\mathrm{G},g,q}(\mathcal{A}) \coloneqq \Pr[\mathrm{GDH}_{\mathrm{G},g,q}(\mathcal{A}) \Rightarrow 1].$$

Game $\mathrm{GDH}_{\mathrm{G},g,p}(\mathcal{A})$:
00 $CR \leftarrow \emptyset$
01 $(i, j, Z) \leftarrow\$ \mathcal{A}$
02 If $\{i, j\} \subseteq [n] \setminus CR \wedge g^{x_i x_j} = Z$:
03 Stop with 1
04 Stop with 0

Oracle Gen
05 $x_n \leftarrow\$ \mathbb{Z}_p^*$
06 $n \leftarrow n + 1$
07 Return $g^{x_{n-1}}$

Oracle $\mathrm{DH}(i, Y, Z)$
08 Require $i \in [n]$
09 If $Y^{x_i} = Z$:
10 Return `tru`
11 Return `fal`

Oracle $\mathrm{Corrupt}(i)$
12 Require $i \in [n]$
13 $CR \xleftarrow{\cup} \{i\}$
14 Return x_i

Figure 22. Multi-instance and corruptable Gap Diffie-Hellman security game for a cyclic group G with generator g of order p.

E.6. Key Encapsulation Mechanism

Syntax. A key encapsulation mechanism KEM consists of three algorithms, KEM.Gen, KEM.Enc and KEM.Dec with the following syntax:

- $\mathrm{KEM.gen} : \emptyset \rightarrow_{\$} \mathcal{SK} \times \mathcal{PK}$
- $\mathrm{KEM.enc} : \mathcal{PK} \rightarrow_{\$} \mathcal{K} \times \mathcal{C}$
- $\mathrm{KEM.dec} : \mathcal{SK} \times \mathcal{C} \rightarrow \mathcal{K}$

Definition 12. *Consider the game* CORR *in Figure 23. We define the correctness error of KEM scheme* KEM *as*

$$\mathrm{Adv}^{\mathrm{corr}}_{\mathrm{KEM}}(\mathcal{A}) \coloneqq \Pr[\mathrm{CORR}_{\mathrm{KEM}}(\mathcal{A}) \Rightarrow 1].$$

Game $\mathrm{CORR}_{\mathrm{KEM}}(\mathcal{A})$
00 Invoke $\mathcal{A}$
01 Stop with 0

Oracle Gen
02 $(\mathrm{sk}, \mathrm{pk}) \leftarrow\$ \mathrm{KEM.gen}$
03 Return $(\mathrm{sk}, \mathrm{pk})$

Oracle Enc
04 $(k, c) \leftarrow\$ \mathrm{KEM.enc}(\mathrm{pk})$
05 $k' \leftarrow \mathrm{KEM.dec}(\mathrm{sk}, c)$
06 If $k \neq k'$: Stop with 1
07 Return (k, c)

Figure 23. Correctness game CORR for KEM scheme KEM.

Security. We define collision resistance as a weak notion for KEM schemes which state that an adversary who has the KEM secret key should not be able to produce the same symmetric key in different encapsulations. This definition extends the LEAK-BIND-K,PK-CT property of [53] which states that for fixed symmetric key-public key instances, there are no collisions in the ciphertexts. Our notion additionally requires that ciphertexts never repeat.

Additionally, we require IND-CCA security for KEMs which means that and adversary against an adversary against a multi-instance KEM game with adaptive corruptions cannot distinguish whether a challenge encapsulation contains the real shared key output by the KEM or a uniformly random key.

Definition 13. *Consider the game* COLL *in Figure 24. We define the advantage of an adversary* $\mathcal{A}$ *against the multi-instance and exposable indistinguishability under chosen ciphertexts of a KEM scheme* KEM *as*

$$\mathrm{Adv}^{\mathrm{coll}}_{\mathrm{KEM}}(\mathcal{A}) \coloneqq \Pr[\mathrm{COLL}_{\mathrm{KEM}}(\mathcal{A}) \Rightarrow 1].$$

Game $\mathrm{COLL}_{\mathrm{KEM}}(\mathcal{A})$
00 $K \leftarrow \emptyset$
01 $\mathrm{pk} \leftarrow\$ \mathcal{A}$
02 Require $\mathrm{pk} \in \mathcal{PK}$
03 Invoke $\mathcal{A}$
04 Stop with 0

Oracle Enc
05 $(k, c) \leftarrow\$ \mathrm{KEM.enc}(\mathrm{pk})$
06 If $k \in K$: Stop with 1
07 $K \xleftarrow{\cup} \{k\}$
08 Return (k, c)

Figure 24. Collision resistance security game COLL for KEM scheme KEM.

Definition 14. *Consider the game* IND-CCA *in Figure 25. We define the advantage of an adversary* $\mathcal{A}$ *against the multi-instance and exposable indistinguishability under chosen ciphertexts of a KEM scheme* KEM *as*

$$\mathrm{Adv}^{\mathrm{ind\text{-}cca}}_{\mathrm{KEM}}(\mathcal{A}) \coloneqq \left|\Pr[\mathrm{IND\text{-}CCA}^0_{\mathrm{KEM}}(\mathcal{A}) - \Pr[\mathrm{IND\text{-}CCA}^1_{\mathrm{KEM}}(\mathcal{A})]\right|.$$

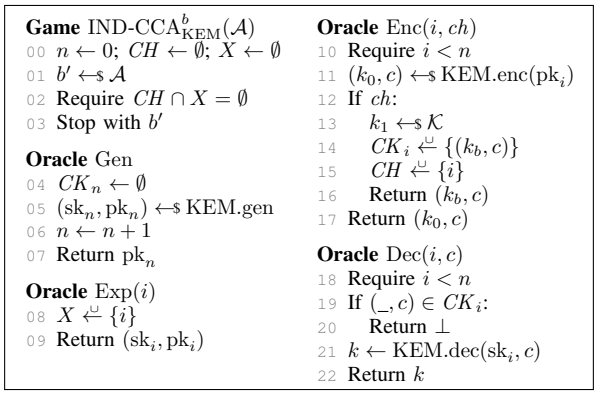

Figure 25. Multi-instance and exposable IND-CCA security game for KEM scheme KEM.

Appendix F. Pseudocode of Receiver for Figure 1

In Figure 26, we provide the recv algorithms corresponding to the send algorithms provided in Figure 1.

Appendix G. Further Discussions on SAKE Model

One-Time Prekeys. SAKE does not capture one-time prekeys, which are used by Signal and have been modeled in previous work. In Signal, a user at registration time first uploads a batch of 100 one-time keys, and re-uploads whenever it exhausts most (all but 10) of its keys. By contrast, semi-static prekeys, which we model as the prekey bundle key, are rotated every 2 days, which can be much shorter than the re-upload time, i.e., provides greater forward secrecy than some one-time keys.

Since one-time keys are not signed, it is well-known that X3DH and PQXDH only satisfy *weak forward secrecy*, since the adversary can take a prekey bundle, replace the one-time key with its own and inject to the sender, compromise the receiver's semi-static and long-term keys and then learn the session key, which it should not be able to do for optimal security. To not overcomplicate our model and because they have been analyzed in previous work [12], we do not formalize them but believe our formalism can be extended to capture them if desired.

Randomness Corruption. Our model does not allow the attacker to compromise or influence the randomness of KE.send or KE.genSPK (though it can expose the corresponding keys). The NAXOS trick [54] can be used generically to protect KE.genSPK/KE.send calls if either the randomness or caller's identity secret key are corrupted (but not both);[6] see [30, Section 1.3] for some further discussion on it. To capture this attack vector, we would have to introduce new trivial attacks: the above should capture baseline attacks. For SealedPQXDH, since there is no 'long-term'-'long-term' Diffie-Hellman computation, without the NAXOS trick, the attacker must be restricted to not simultaneously to corrupt the sender's randomness and the receiver's ssk (i.e., compromise both short-term keys); this is not a problem in XHMQV. [Daniel: sketch or even formalize new trivial attacks? make slightly more transparent in the main body that we dont explicitly address] [Daniel: mention that our MS-KEM game somehow supports 'more' than we do for SAKE?]

6. We refer to randomness *exposure* here, not randomness *manipulation* [55] which is harder to defend against.

Testing. Our key indistinguishability requires the adversary commit to whether a session key is challenged or not in the Send call that the key is produced. Some notions in the literature allow the attacker to later decide to challenge these keys. We believe that our KIND notion reduces to a definition that allows this albeit non-tightly. As a result, existing composability results on Bellare-Rogaway-style AKE models [56] should morally apply for our definition, too [57]. We prefer to adopt our notion as we believe it is sufficient and captures the fact that session keys should be secure immediately when they are used.

Authentication. SAKE models sender authentication by requiring the receiver to input a set of public keys to KE.recv. In practice, this mechanism could be implemented differently, for example if the sender sends a long-term key to the receiver and a signed certificate from the public-key infrastructure containing say a hash of the long-term key. Note that in SealedRingXKEM-XHMQV that the sender sends their long-term ring signature and DH identity key to the receiver. In practice, this means that the PKI query can be 'deferred' to the sender, reducing latency on the receiver side.

Appendix H. Proofs for IND Security

In this section, we prove IND security for MK instances $\mathrm{MK}^1_{\mathrm{p}}, \mathrm{MK}^2_{\mathrm{p}}, \mathrm{MK}^1_{\mathrm{h}}$, and $\mathrm{MK}^2_{\mathrm{h}}$. Since both subconstructions in $\mathrm{MK}^1_{\mathrm{p}} = (\mathrm{MK}^1_{\mathrm{p}}, \mathrm{MK}^2_{\mathrm{p}})$ (and, respectively, $\mathrm{MK}^1_{\mathrm{p}} = (\mathrm{MK}^1_{\mathrm{p}}, \mathrm{MK}^2_{\mathrm{p}})$) use the receiver's long term key, we prove the IND security jointly. Intuitively, the MS-KEM IND security means that, given pair of encapsulation and key (k, c), the adversary cannot distinguish whether the key is real encapsulated key or a randomly drawn one; as long as one of the keys input to the encapsulations remains uncorrupted.

Proof Strategy: Case Distinctions. The subconstructions follow a similar structure: First, intermediate shared secrets are established through DH key agreements and KEM encapsulations. These intermediate secrets are then put into a KDF, which we model as random oracle (RO) which outputs the final shared key k. Since the output of the RO is random per definition, the adversary can only distinguish the real key k from a random one if the adversary knows

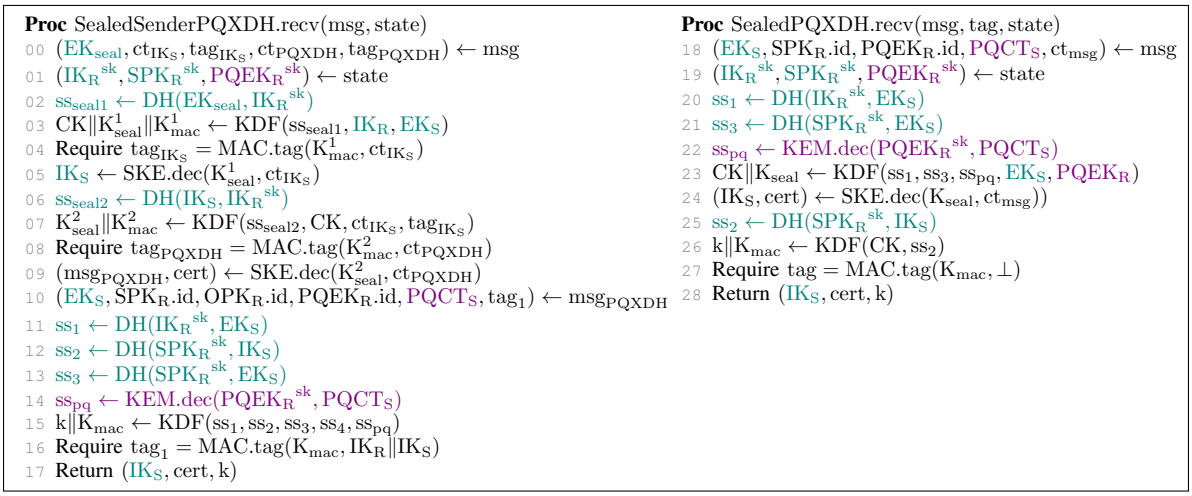

Figure 26. Comparison of simplified recv functions for SealedPQXDH and Sealed Sender + PQXDH.

the intermediate secrets. Now, we know that one of these intermediate secrets is secure, i.e., unknown to the adversary since the IND security definition requires that at least one key pair that was put into the MS-KEM and used to derive an intermediate secret is uncorrupted. Therefore, in the end we can reduce the security of the output key k with case distinctions to the security of the underlying primitive.

Additionally, observe that not all challenge queries are covered by all primitives equally: The primitive's type determines whether it provides security to challenge encapsulations, decapsulations or both. That is, type `r` (here: a post-quantum KEM) protects encapsulation challenges, `s` (e.g., signatures) protects decapsulations challenges, and type `b` (here: DH) and the pre-shared key protect both.

Definition 15. *Let $\mathcal{B}_1$ be an adversary against the collision resistance* COLL *of KEM scheme* KEM*, $\mathcal{B}_2$ be an adversary against* IND-CCA *security against of KEM scheme* KEM*, let $\mathcal{B}_3$ be an adversary against* GDH *security with group* G *of order p, and generator g. Let q_H denote the number of queries from the adversary to the RO, $q_{\mathrm{ch}}^{\mathrm{enc}}$ resp. $q_{\mathrm{ch}}^{\mathrm{dec}}$ be the adversary's encapsulation resp. decapsulation challenge queries with $q_{\mathrm{ch}} := q_{\mathrm{ch}}^{\mathrm{enc}} + q_{\mathrm{ch}}^{\mathrm{dec}}$, and q_{enc} resp. q_{psk} be the number of encapsulation resp. pre-shared key generation queries with pre-shared key space $\mathcal{PSK}$, and KEM symmetric key space $\mathcal{K}_{\mathrm{KEM}}$.*

H.1. SealedPQXDH: $\mathrm{MK}_{\mathrm{p}}^1$ and $\mathrm{MK}_{\mathrm{p}}^2$

We show that $\mathrm{MK_p}$ from Figure 2 and Figure 7 satisfies the IND security definition from Figure 9 as denoted in Theorem 1.

Theorem 6. *Consider the adversaries, variables, and definitions from Definition 15, let $T[\mathtt{dh}] = \mathtt{b}, T[\mathtt{pqkem}] = \mathtt{r}$. The advantage of an adversary $\mathcal{A}$ against the* IND *security of MS-KEM scheme $\mathrm{MK_p} = (\mathrm{MK}_{\mathrm{p}}^1, \mathrm{MK}_{\mathrm{p}}^2)$ is as follows.*

$$\mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK_p},T}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{coll}}_{\mathrm{KEM}}(\mathcal{B}_1) + \frac{q_{\mathrm{psk}}}{|\mathcal{PSK}|} + \max\left(\frac{q_H q_{\mathrm{psk}}}{|\mathcal{PSK}|}, \mathrm{Adv}^{\text{ind-cca}}_{\mathrm{KEM}}(\mathcal{B}_2), \mathrm{Adv}^{\mathrm{gdh}}_{\mathrm{G},g,p}(\mathcal{B}_3)\right)$$

Game 0. We start with the security game from Figure 9 with the following bound.

$$\mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}) = \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK},T}(\mathcal{A}).$$

Game 1. We start by introducing lazy sampling for the KDF which we model as RO. That means, we initialize the RO as an empty lookup table and whenever the RO is queried with a new value, we randomly sample a value, store it, and return it. Otherwise, if the RO receives a repeated query, it returns the same value as before. Since this only changes the time of computation but nothing else, this change is indistinguishable to the adversary.

$$\mathrm{Adv}^{\mathrm{G1}}(\mathcal{A}) = \mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}).$$

Game 2. In this game, we prohibit KEM key collisions. This is a weak form of security where the adversary is allowed to know the KEM's secret key. That means, we keep track of the ciphertexts produced in the encapsulations ($(c, k) \leftarrow\$ \mathrm{KEM.enc}(\mathrm{pk})$) and abort if is produces two pairs (c, k) and (c', k) with $c \neq c'$. Similarly, we abort if in the decapsulation, two different ciphertexts $c \neq c'$ decapsulate to the same key, i.e., $k \leftarrow \mathrm{KEM.dec}(\mathrm{sk}, c) = \mathrm{KEM.dec}(\mathrm{sk}, c')$. An adversary notices this change if it breaks the collision resistance of the KEM scheme. Let $\mathcal{B}_1$ be an adversary against the collision resistance of the KEM scheme. Then,

$$\mathrm{Adv}^{\mathrm{G1}}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{coll}}_{\mathrm{KEM}}(\mathcal{B}_1) + \mathrm{Adv}^{\mathrm{G2}}(\mathcal{A}).$$

Game 3. In this game, we abort if the game draws the same pre-shared key twice. The probability for detecting

this change is bounded by the birthday bound in q_{psk} pre-shared key generation queries and the pre-shared key space $\mathcal{PSK}$:

$$\mathrm{Adv}^{\mathrm{G2}}(\mathcal{A}) \leq \frac{q_{\mathrm{psk}}}{|\mathcal{PSK}|} + \mathrm{Adv}^{\mathrm{G3}}(\mathcal{A}).$$

Game 4. Now, we adapt the RO such that the game and the adversary have different RO interfaces. The game has access to an internal interface RO and the adversary to a special adversarial interface $\mathrm{RO}_{\mathcal{A}}$. Whenever the game internally calls the RO to derive the final key, it provides full context by inputting the keys which were input to MK as associated data. With this change, the game stops submitting the intermediate secrets, i.e., DH and KEM shared secrets except the pre-shared key, to the RO, but obtains the resulting key only through the context. On the other side, if the adversary wants to obtain the same key through the RO, they have to submit the full context and additionally the shared secrets to $\mathrm{RO}_{\mathcal{A}}$. Only if the shared secrets are correct, the adversary obtains the same values which the game computed; otherwise, the adversary obtains a randomly drawn value. In the following, we show that if the adversary, without corrupting *all* key pairs, is able to query the RO at an entry which the game internally used, then the adversary breaks the security of one of the underlying primitives.

With the following case distinction, we show for each primitive (pre-shared key, DH, PQ-KEM) how security can be reduced to it if the corresponding key (pair) remains uncorrupted.

Case 1: Pre-shared key uncorrupted. If the pre-shared key is uncorrupted, the adversary wins if it correctly guesses this randomly drawn key. In this game, we abort if the adversary queries the RO with an input that includes an uncorrupted pre-shared key which the game uses for an encapsulation or decapsulation. The output of the RO appears to the adversary as a random key as long as they do not query the RO with the pre-shared key.Since the pre-shared key is random and uniformly sampled, the probability that the adversary guesses the secret key is bounded in the birthday bound by the number of pre-shared keys, i.e., number of calls to generate pre-shared keys, q_{psk}, the number of RO queries from the adversary q_H, and the pre-shared key space $\mathcal{PSK}$. At the end, the challenge key returned from the RO is random and the adversary has no advantage in winning the game.

$$\mathrm{Adv}^{\mathrm{G3}}(\mathcal{A}) \leq q_H \cdot \frac{q_{\mathrm{psk}}}{|\mathcal{PSK}|} + \mathrm{Adv}^{\mathrm{G4.C1}}(\mathcal{A}).$$

Case 2: DH key uncorrupted. In this case, the two DH keys used for a challenge encapsulation or decapsulation remain uncorrupted and we show that this yields security for the resulting output key. For that, in this game hop, we abort whenever the adversary queries the RO with a solution to GDH as described below. That is, we show that if the adversary queries the RO at the input for a (challenge) key, we can build a reduction from this adversary which breaks GDH (Figure 22). The reduction is as follows:

- $\mathrm{MK.Gen}$: First, we forward all DH key generation queries to the multi-instance GDH challenger and forward the returned public keys to the adversary. If the adversary registers their own key material, this is stored and indexed by the reduction.
- Corrupt: Whenever the adversary corrupts any of the internal DH keys, we corrupt the instance of the GDH challenger and return it to the adversary. We note that as soon as the adversary corrupts one of the GDH instances used for the challenge, the adversary can neither win anymore in the IND game (nor produce a valid result for that instance in the GDH game).
- $\mathrm{MK.Enc/Dec}$: Next, we consider how encapsulation and decapsulation queries are handled. Our first observation is that the DH and KEM (key) material determines uniquely the query to the RO (recall, we ruled out KEM collisions in **Game 2**), including which DH combinations were used to generate the intermediate secrets. When the adversary calls the (challenge) encapsulation or decapsulation oracle, the reduction first performs the KEM operation as specified in the algorithm. The reduction then queries the RO at $\mathrm{RO}(\mathrm{ad})$ for the output key k, ommitting the intermediate secrets.
- $\mathrm{RO}_{\mathcal{A}}$: Now, we consider how the adversary interacts with the RO. First, we require the adversary to ask full queries to the RO (different to the reduction, which may leave parts of the query undefined). That is, it must provide ad (public keys), potentially $\mathrm{ss}_{\mathrm{pq}}$ (the KEM secret which the adversary can learn by creating a ciphertext itself or corrupting the respective KEM key) and a pre-shared key psk, and the value(s) ss (DH shared secrets), all of which are $\neq \bot$. We map the given DH keys $\mathrm{pk}_i \in \mathrm{ad}_1$ to their (potential) GDH or adversarially given instances i.
  For each DH pair (i, j), we do the following: If neither key pair maps to a GDH instance, this pair cannot be used to solve GDH and we continue (i.e., we leave this in the RO query but do not forward it to the GDH challenger). If one of i and j correspond to a GDH instance, we forward the query to GDH as follows. Assume i (resp. j) corresponds to a GDH instance and j (resp. i) marks the (other) public key pk, then we forward $(i, \mathrm{pk}, \mathrm{ss})$ (resp. $(j, \mathrm{pk}, \mathrm{ss})$ to the GDH oracle. For the adversary to provide the correct ss, i.e., $\mathrm{DH}(i, \mathrm{pk}, \mathrm{ss})$=`tru`, we have the following cases: (a) If i or j is corrupted or corresponds to an adversarially inserted key, the adversary can trivially compute the corresponding ss. Note here that the adversary automatically cannot win the IND game anymore if it corrupted or set all DH key pairs of a challenge encapsulation/decapsulation. In this case, the reduction internally computes whether the provided ss is correct, and if it is, returns the game's internally computed RO output. (b) Otherwise, both i and j correspond to GDH instances and the adversary did not learn the corresponding private keys (x_i, x_j) but provided a solution for $g^{x_i x_j} = \mathrm{ss}$. This is therefore a solution to GDH and our game aborts if (i, j) correspond to the

DH keys for the challenge query.
Lastly, if the adversary provided a wrong DH computation, we create a new RO entry from the adversary's RO query and sample a random output.

Overall, the adversary can only query the RO for the input that generates the output key k if it breaks GDH security. Otherwise, the key returned by the game is random and the adversary has no advantage for guessing the correct bit b. Let $\mathcal{B}_3$ be an adversary against the GDH security (Figure 22). The advantage of an adversary $\mathcal{A}$ against this game is then bounded by

$$\mathrm{Adv}^{\mathrm{G3}}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{gdh}}_{\mathrm{G},g,p}(\mathcal{B}_3) + \mathrm{Adv}^{\mathrm{G4.C2}}.$$

Case 3: PQ-KEM key uncorrupted. In this game, we prevent the adversary from querying the RO with a decapsulated key k_{KEM} for a KEM ciphertext which $\mathrm{MK.enc}$ produced and for which they do not know the corresponding secret key. That is, we abort whenever the adversary queries $\mathrm{RO}_{\mathcal{A}}$ with a query that includes a KEM ciphertext generated as output of the $\mathrm{MK.Enc}$ oracle and its valid decapsulated key $(k_{\mathrm{KEM}}, c_{\mathrm{KEM}})$ for which the adversary did not set or corrupt the secret key $\mathrm{sk}_{\mathrm{KEM}}$. We bound the probability of this event by reducing to the OW-KCA security of the underlying KEM scheme. In other words, if the adversary detects this change, they break the KEM's OW-KCA security.

First, observe that the key indistinguishability of decapsulation challenges is not protected through the KEM and the KEM key may be corrupted. (The adversary knows the receiver's public key and can internally compute a KEM ciphertext for which it knows the key to input to the decapsulation oracle.) That means, if this is a decapsulation challenge, security reduces to the remaining DH primitive (see **Case 2**).

The reduction is as follows.

- $\mathrm{MK.Gen}$: We forward the KEM key generation queries to the multi-instance KEM challenger. The KEM keys which the adversary registers are indexed and stored by the reduction as before. Note that, per definition, any decapsulation query requires that a secret key is registered.
- $\mathrm{MK.Corrupt}$: If the adversary corrupts a KEM key, we corrupt the corresponding KEM instance and forward the key to the adversary. In case the key was registered by the adversary itself, we return their keys.
- $\mathrm{MK.Enc}$: When the adversary queries the encapsulation oracle, the reduction forwards the KEM encapsulation to the respective KEM instance and embeds the returned ciphertext c_{KEM} as context for the KDF and in the returned key-ciphertext pair. We mark c_{KEM} as internally generated.
- $\mathrm{MK.Dec}$: Next, when the game calls the KEM decapsulation, the reduction internally leaves this value empty and, as described in Game 4's setup, queries the RO only with the context given through the KEM ciphertext. Note that the KEM security does not protect decapsulation challenges (implicit authentication) because the adversary can freely produce encapsulations towards the receiver's public key and thereby know the corresponding secret key. (This means security here is given through the remaining primitives.) Consistency for the RO is ensured as described in the next item.
- $\mathrm{RO}_{\mathcal{A}}$: When the adversary queries the RO for a key, we check whether the KEM ciphertext c_{KEM} was generated by querying the KEM challenger's encapsulation. If not, we sample a fresh value or return an already sampled entry. If the ciphertext stems from the KEM challenger and the challenger's key was corrupted, the RO internally checks whether the key is a valid decapsulation of the ciphertext and if so, returns the game's internal value. If the challenger's is not corrupted, we check through the challenger's Check oracle whether the provided key is a valid encapsulation and abort if it is true. Otherwise, we return a randomly sampled value.

This concludes the reduction. The adversary thereby detects this change if they break the OW-KCA security of the underlying KEM scheme. Let $\mathcal{B}_2$ be an adversary against the KEM scheme. The advantage of an adversary for distinguishing this and the previous game is then bounded by

$$\mathrm{Adv}^{\mathrm{G3}(\mathcal{A})} \leq \mathrm{Adv}^{\mathrm{ind\text{-}cca}}_{\mathrm{KEM}}(\mathcal{B}_2) + \mathrm{Adv}^{\mathrm{G4.C3}}.$$

Randomized encapsulation and decapsulation. The variable R indicates whether the encapsulation and decapsulation is deterministic or includes random values.

In $\mathrm{MK}^2_{\mathrm{p}}$, encapsulation only relies on Diffie–Hellman and pre-shared keys and is therefore deterministic. Consequently, repeated challenge queries with identical key material would yield identical outputs, allowing an adversary to trivially distinguish real from random. Such repeated queries must therefore be disallowed.

In contrast, $\mathrm{MK}^1_{\mathrm{p}}$ incorporates a probabilistic PQ-KEM. Each encapsulation produces fresh randomness, resulting in a new ciphertext and session key even when the underlying key material is unchanged. As a result, the input to the random oracle is fresh for every encapsulation and for every distinct decapsulation query. Even if the adversary repeatedly requests encapsulations under the same key material and subsequently corrupts the KEM secret key, distinguishing remains infeasible. Although the DH component is deterministic, the randomness introduced by the KEM ensures that each random oracle query is independent. Hence, the oracle outputs appear uniform and independent, preventing the adversary from distinguishing real from random keys.

Wrapping up the proof. Finally, the output of challenge encapsulations and decapsulations is a random value output by the RO. This value is indistinstuishable for the adversary because it never queries the RO at the respective RO input through the above case distinction. The advantage of the final game in all cases is thus

$$\mathrm{Adv}^{\mathrm{G4}}(\mathcal{A}) = 0.$$

The first construction $\mathrm{MK}^1_{\mathrm{p}}$ uses two DH key pairs $\{(\mathrm{EK_S}, \mathrm{IK_R}), (\mathrm{EK_S}, \mathrm{SPK_R})\}$ and a PQ-KEM key $\mathrm{PQEK_R}$.

The second construction $\mathrm{MK}_{\mathrm{p}}^2$ uses a DH key pair $(\mathrm{IK}_{\mathrm{S}}, \mathrm{SPK}_{\mathrm{R}})$ and a pre-shared key CK. Our security definition states that the MK construction must be secure as long as at least one key (pair) is uncorrupted. Specifically, security for encapsulation challenges is given as long as one of the above DH key pairs or the PQ-KEM key/pre-shared key is uncorrupted. Security for decapsulation challenges is given as long as one of the above DH key pairs or the pre-shared key is uncorrupted.

This means that the adversary can corrupt key freely as long as a certain core of keys remains uncorrupted. To distinguish real and random challenges, the adversary then needs to break one of these core primitives. In the following, we consider which key combinations build a core.

With slight abuse of notation, let $\mathrm{EK}_{\mathrm{S}}, \ldots$ be the event that $\mathrm{EK}_{\mathrm{S}}, \ldots$ is uncorrupted. For each challenge, we can now define the necessary conditions for a challenge to not be a trivial win. That is, let $E^{p,\{1,2\}}_{\{\mathrm{Enc},\mathrm{Dec}\}}$ be the event that a challenge encapsulation/decapsulation in construction $\mathrm{MK}_{\mathrm{p}}^1/\mathrm{MK}_{\mathrm{p}}^2$ is not a trivial win, but there are enough uncorrupted keys.

$$
\begin{aligned}
E^{p,1}_{\mathrm{Enc}} &= \mathrm{EK}_{\mathrm{S}} \wedge \mathrm{IK}_{\mathrm{R}} \vee \mathrm{EK}_{\mathrm{S}} \wedge \mathrm{SPK}_{\mathrm{R}} \vee \mathrm{PQEK}_{\mathrm{R}},\\
E^{p,1}_{\mathrm{Dec}} &= \mathrm{EK}_{\mathrm{S}} \wedge \mathrm{IK}_{\mathrm{R}} \vee \mathrm{EK}_{\mathrm{S}} \wedge \mathrm{SPK}_{\mathrm{R}},\\
E^{p,2}_{\mathrm{Enc}} &= \mathrm{IK}_{\mathrm{S}} \wedge \mathrm{SPK}_{\mathrm{R}} \vee \mathrm{CK}, \text{and}\\
E^{p,2}_{\mathrm{Dec}} &= \mathrm{CK}.
\end{aligned}
$$

The above conjunctions build the cores, i.e., minimal combinations of primitives which need to be uncorrupted s.t. there is no trivial win.

To break the security, an adversary would need to break any one of the primitives in the conjunctions. To obtain the highest advantage, the adversary would (a) attempt to distinguish the game by breaking key indistinguishability or implicit authentication based on which is easier, and (b) choose the primitive with the highest advantage term. For the final advantage, we therefore calculate the bound by taking the maximum over all used primitives.

H.2. SealedRingXKEM-XHMQV: MK_{h}

We prove the IND security of the MK_{h} construction from Figure 2 and Figure 7 as denoted in Theorem 1.

Theorem 7. *Consider the adversaries, variables, and definitions from Definition 15, let* $T[\mathtt{dh}] = \mathtt{b}, T[\mathtt{pqkem}] = \mathtt{r}$. *The advantage of an adversary* $\mathcal{A}$ *against the* IND *security of MS-KEM scheme* $\mathrm{MK}_{\mathrm{h}} = (\mathrm{MK}_{\mathrm{h}}^1, \mathrm{MK}_{\mathrm{h}}^2)$ *is as follows.*

$$
\begin{aligned}
&\mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK}_{\mathrm{h}},T}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{coll}}_{\mathrm{KEM}}(\mathcal{B}_1) + \frac{q_{\mathrm{psk}}}{|\mathcal{PSK}|}\\
&+ \max\left(\frac{q_H q_{\mathrm{psk}}}{|\mathcal{PSK}|}, \mathrm{Adv}^{\text{ind-cca}}_{\mathrm{KEM}}(\mathcal{B}_2), \mathrm{Adv}^{\mathrm{gdh}}_{\mathrm{G},g,p}(\mathcal{B}_3)\right)
\end{aligned}
$$

Proving IND security for the construction MK_{h} from Figure 2 and Figure 7 almost directly follows from the proof in Appendix H.1. We start with lazy sampling and bounding collision resistance for the KEM and pre-shared keys. Since both KEM operations use the same construction, this reduces to a single advantage term against the collision resistance of the KEM. That is, the $\mathrm{MK}_{\mathrm{h}}^1$ en- and decapsulation takes as input two DH pairs and two KEM keys (notably, for different KEM constructions), and we can perform a hybrid argument over all challenge queries and a similar case distinction as before. Per IND definition, for a challenge encapsulation, one of the DH pairs and KEM keys needs to be uncorrupted. For challenge decapsulations, one of the DH pairs needs to be uncorrupted.

To reduce to GDH in $\mathrm{MK}_{\mathrm{h}}^1$, we extend Line 27 and Line 12 to

$$
\begin{aligned}
\mathrm{ss}_3 \leftarrow (\mathrm{IK}_{\mathrm{R}}{}^{e_1}\mathrm{SPK}_{\mathrm{R}})^{\mathrm{EK}_{\mathrm{S}}{}^{\mathrm{sk}}}&\\
= (\mathrm{IK}_{\mathrm{R}}{}^{e_1})^{\mathrm{EK}_{\mathrm{S}}{}^{\mathrm{sk}}}(\mathrm{SPK}_{\mathrm{R}})^{\mathrm{EK}_{\mathrm{S}}{}^{\mathrm{sk}}}&\\
= (\mathrm{EK}_{\mathrm{S}}{}^{\mathrm{IK}_{\mathrm{R}}{}^{\mathrm{sk}}})^{e_1}(\mathrm{EK}_{\mathrm{S}}{}^{\mathrm{SPK}_{\mathrm{R}}{}^{\mathrm{sk}}})&\\
= \mathrm{EK}_{\mathrm{S}}{}^{e_1\mathrm{IK}_{\mathrm{R}}{}^{\mathrm{sk}}+\mathrm{SPK}_{\mathrm{R}}{}^{\mathrm{sk}}}.&
\end{aligned}
$$

We see that this is a product two DH operations with an additional exponentiation with public value e_1. As long as one of the factors is secure, ss_3 is secure as well. For the hybrid games, we abort if the adversary queries the RO at the DH keys used for the challenges and provides a solution for the underlying GDH instances. We follow exactly the reduction in Appendix H.1 to show that the adversary cannot know ss_3 as long as the DH keys in one of the factors is uncorrupted and the adversary cannot break GDH.

To reduce to GDH in $\mathrm{MK}_{\mathrm{h}}^2$, we extend the DH operations at Lines 31 and 16 to

$$
\begin{aligned}
\mathrm{ss}_4 \leftarrow (\mathrm{IK}_{\mathrm{R}}{}^{e_1}\mathrm{SPK}_{\mathrm{R}})^{d\mathrm{IK}_{\mathrm{S}}{}^{\mathrm{sk}}}&\\
=(\mathrm{IK}_{\mathrm{R}}{}^{e_1})^{d\mathrm{IK}_{\mathrm{S}}{}^{\mathrm{sk}}}(\mathrm{SPK}_{\mathrm{R}})^{d\mathrm{IK}_{\mathrm{S}}{}^{\mathrm{sk}}}&\\
=(\mathrm{IK}_{\mathrm{S}}{}^{\mathrm{IK}_{\mathrm{R}}{}^{\mathrm{sk}}})^{e_1 d}(\mathrm{IK}_{\mathrm{S}}{}^{\mathrm{SPK}_{\mathrm{R}}{}^{\mathrm{sk}}})^{d}&\\
=(\mathrm{IK}_{\mathrm{S}}{}^{d})^{e_1\mathrm{IK}_{\mathrm{R}}+\mathrm{SPK}_{\mathrm{R}}}.&
\end{aligned}
$$

We see that this is a product two DH operations with an additional exponentiation with public values e_1 and d. As long as one of the factors is secure, ss_4 is secure as well. For the hybrid games, we abort if the adversary queries the RO at the DH keys used for the challenges and provides a solution for the underlying GDH instances. We follow exactly the reduction in Appendix H.1 to show that the adversary cannot know ss_3 as long as the DH keys in one of the factors is uncorrupted and the adversary cannot break GDH.

With this case distinction, and considering that the adversary may corrupt the keys which give them the largest advantage, we arrive at the resulting bound.

Appendix I. Proofs for ANON Security

In this section, we present the proofs showing that the constructions from Section 2.1 satisfy the ANON security definition from Figure 10 as denoted in Theorem 1. We use the variables and advantages from Definition 15 in the following.

I.1. SealedPQXDH: MK_p^1

Theorem 8. *Consider the adversaries, variables, and definitions from Definition 15. The advantage of an adversary $\mathcal{A}$ against the* ANON *security of MS-KEM scheme $\mathrm{MK}_\mathrm{p} = (\mathrm{MK}_\mathrm{p}^1, \mathrm{MK}_\mathrm{p}^2)$ is as follows.*

$$\mathrm{Adv}^{\mathrm{anon}}_{\mathrm{MK_p},T}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{coll}}_{\mathrm{KEM}}(\mathcal{B}_1) + \frac{q_{\mathrm{psk}}}{|\mathcal{PSK}|} \max\left(\mathrm{Adv}^{\mathrm{gdh}}_{\mathrm{G},g,p}(\mathcal{B}_3), \frac{q_{\mathrm{psk}} q_H}{|\mathcal{PSK}|}\right)$$

We show that construction MK_p from Figure 2 and Figure 7 satisfies the anonymity definition from Figure 10. The definition states that the returned key-ciphertext pair (k, c) does not reveal anything about the sender. First, observe that in this construction, c is generated only with receiver key material (Line 16), and therefore sender anonymous, it thereby suffices to show that k also does not reveal the sender identity. The first part of the construction has a randomized encapsulation ($R = 1$) which relies on collision-free outputs of the inner KEM encapsulation. Proving sender anonymity of the returned k basically uses the fact that the returned key is indistinguishable from random; thereby the following proof sketches the steps from Appendix H.

Game 0. We start with the security game from Figure 10.

$$\mathrm{Adv}^{\mathrm{G0}}_{\mathcal{A}} = \mathrm{Adv}^{\mathrm{anon}}_{\mathrm{MK},T,\mathcal{A}}.$$

Game 1. First, we introduce lazy sampling for the random oracle. Now, values are randomly sampled and stored at the time they are queried. Since this only changes the time of computation, the adversary cannot notice the difference.

$$\mathrm{Adv}^{\mathrm{G1}}_{\mathcal{A}} = \mathrm{Adv}^{\mathrm{G0}}_{\mathcal{A}}$$

Game 2. In this game, we stop if the inner KEM encapsulation ever returns the same key. This ensures that the input to the RO is different with each MS-KEM encapsulation. We reduce this to the collision resistance of the underlying KEM scheme as in Appendix H.1. Let $\mathcal{B}_1$ be an adversary against the collision resistance of the KEM scheme. Then,

$$\mathrm{Adv}^{\mathrm{G1}}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{coll}}_{\mathrm{KEM}}(\mathcal{B}_1) + \mathrm{Adv}^{\mathrm{G2}}(\mathcal{A}).$$

Game 3. In this game, we abort if the game draws the same pre-shared key twice. The probability for detecting this change is bounded by the birthday bound in q_{psk} pre-shared key generation queries and the pre-shared key space $\mathcal{PSK}$:

$$\mathrm{Adv}^{\mathrm{G2}}(\mathcal{A}) \leq \frac{q_{\mathrm{psk}}}{|\mathcal{PSK}|} + \mathrm{Adv}^{\mathrm{G3}}(\mathcal{A}).$$

Game 4. Now, we do the switch to providing the adversary with a special access to the RO through the interface $\mathrm{RO}_{\mathcal{A}}$. To derive the final key, the game now internally does not provide the intermedia shared secrets to the RO anymore but only the context. If the adversary wants to obtain the same value as the game internally computed, they have to query the RO with the context and all correct intermediate values. As in Appendix H, we can show that this reduces to the security of the pre-shared key, GDH, or OW-KCA of the KEM. Since the adversary can freely choose which primitive is easiest to break and corrupt all other primitives, the final advantage is the maximum over all bounds.

I.2. SealedRingXKEM-XHMQV: MK_h

Theorem 9. *Consider the adversaries, variables, and definitions from Definition 15. The advantage of an adversary $\mathcal{A}$ against the* ANON *security of MS-KEM scheme $\mathrm{MK}_\mathrm{h} = (\mathrm{MK}_\mathrm{h}^1, \mathrm{MK}_\mathrm{h}^2)$ is as follows.*

$$\mathrm{Adv}^{\mathrm{anon}}_{\mathrm{MK_h},T}(\mathcal{A}) \leq 2\mathrm{Adv}^{\mathrm{coll}}_{\mathrm{KEM}}(\mathcal{B}_1) + \frac{q_{\mathrm{psk}}}{|\mathcal{PSK}|} \max\left(\mathrm{Adv}^{\mathrm{gdh}}_{\mathrm{G},g,p}(\mathcal{B}_3), \frac{q_{\mathrm{psk}} q_H}{|\mathcal{PSK}|}\right)$$

This proof follows directly from the proofs in Appendix H and Appendix I.1.

Appendix J. On MS-KEM Hybrid Security

Our proofs in Appendices H and I essentially capture that the adversary needs to corrupt or break all security primitives simultaneously to distinguish real from random (IND) or left from right (ANON) in a challenge query. With the case distinctions in the proofs, we capture all possible corruption patterns where at least one key of each challenge remains uncorrupted, as per the security definitions. Regardless of the exact corruption pattern, the adversary which achieves the highest advantage is the one that has to break only the weakest primitive. In other words, the overall advantage is upper-bounded by the *weakest* primitive.

However, we can make this bound more specific based on the exact corruption pattern. More specifically, if the adversary does not corrupt more than one primitive, the resulting advantage is upper-bounded by the *strongest* uncorrupted primitive. In this section, we formalize this observation, which allows us to reason about the *hybrid* security of our key exchange protocols.

First, we define the following advantages

$$\begin{aligned}
\mathrm{Adv}^{\mathtt{pre}}_{\mathrm{KEM}} &:= \mathrm{Adv}^{\mathrm{coll}}_{\mathrm{KEM}}(\mathcal{B}_1) \\
\mathrm{Adv}^{\mathtt{r}}_{\mathrm{KEM}} &:= \mathrm{Adv}^{\text{ind-cca}}_{\mathrm{KEM}}(\mathcal{B}_2), \\
\mathrm{Adv}^{\mathtt{b}}_{\mathrm{G},g,p} &:= \mathrm{Adv}^{\mathrm{gdh}}_{\mathrm{G},g,p}(\mathcal{B}_3), \\
\mathrm{Adv}_{\mathrm{psk}} &:= \frac{q_{\mathrm{psk}}(1+q_H)}{|\mathcal{PSK}|}.
\end{aligned}$$

Structuring IND **and** ANON **Proofs.** We first discuss the general structure of the security proofs for our MK constructions (see Section 2.1). All constructions take as input key pairs (of some fixed primitive) from which they compute secrets. In our constructions, the input pair either is a DH secret and public key which produces a shared secret,

or a KEM public key (or secret key) which encasulates a symmetric key in a ciphertext (or decapsulates a symmetric key from the ciphertext). These secrets are then all combined into a KDF, potentially together with a pre-shared key, to derive the MS-KEM key k. The security argument is then that one of these intermediate keys is secure and we reduce the security to the advantage of an adversary breaking the respective primitive. Additionally, as described in Section 2, not all primitives protect encapsulation and, in the case of IND, decapsulation challenges equally. We thus distinguish between encapsulation and decapsulation challenges when formulating the final bound.

For MS-KEM constructions which are randomized ($R = 1$), i.e., which may be queried for multiple challenges with the same parameters, we additionally need to ensure that the input into the KDF varies with each encapsulation or decapsulation call. In our constructions, this applies to $\mathrm{MK}^1_{\mathrm{p}}$ and $\mathrm{MK}^1_{\mathrm{h}}$, where the only component which may ensure collision-free randomization is the inner KEM (the DH computation and hash operations remain constant across such queries). This means, to bound the advantage of an adversary for winning in IND or ANON, we first need to bound the advantage of an adversary breaking the collision resistance of the KEM, even if the adversary has access to the KEM secret key (but not the encapsulation randomness).

Bounding the Advantage for Specific Patterns of Uncorrupted Primitives. With the above considerations, we now discuss how to bound an adversary's advantage when multiple keys remain uncorrupted. In the following, consider the advantage terms as defined in Definition 15. First, the collision resistance of the KEM is needed for both constructions $\mathrm{MK}^1_{\mathrm{p}}$ and $\mathrm{MK}^1_{\mathrm{h}}$, regardless of which keys remain uncorrupted. We define

$$\mathrm{Adv}^{\mathtt{pre}} := \begin{cases} n_{\mathrm{KEM}} \mathrm{Adv}^{\mathtt{pre}}_{\mathrm{KEM}} & R = 1, \\ 0 & R = 0, \end{cases}$$

where n_{KEM} is the number of KEM invocations in the construction's encapsulation (and decapsulation, resp.) and $\mathrm{Adv}^{\mathtt{pre}}_{\mathrm{KEM}}$ as defined in Definition 15.

Next, we discuss the advantage which an adversary gains from challenge queries if multiple keys are uncorrupted. Intuitively, this means that the adversary needs to break the strongest primitive. More formally, to prove this, we can use the case distinctions from the proofs in Appendices H and I. Instead of letting the adversary freely choose which primitives to corrupt, we fix the uncorrupted primitives in advance. From there, it follows directly that the adversary has to break the security of all uncorrupted primitives simultaneously to win against the IND or ANON game. The advantage of the adversary is then bounded by the *lowest* advantage of all uncorrupted primitives.

For the IND and ANON security, distinguishing between challenge encapsulations and decapsulations, we can now formulate a hybrid argument over all challenge queries which results in the following bound. Let $t \in \{p, h\}$, $i \in \{1, 2\}$, and let $S^i_t \subseteq \{\mathrm{gdh}, \mathrm{kem}, \mathrm{psk}\}$ denote the primitives used in MS-KEM construction MK^i_t, e.g., $S^1_p = \{\mathrm{gdh}, \mathrm{kem}\}$, where gdh stands for a DH key pair, kem for a KEM key, and psk for a pre-shared key. Regarding IND security, for each non-empty set of uncorrupted keys $U^i_t \subseteq S^i_t$, with advantages as defined in Definition 15, it holds that

$$\mathrm{Adv}^{\mathrm{enc}}_{\mathrm{ch,ind}} := q^{\mathrm{enc}}_{\mathrm{ch}} \min \begin{Bmatrix} \mathrm{Adv}^{\mathtt{r}}_{\mathrm{KEM}} & \mathrm{kem} \in U^i_t, \\ \mathrm{Adv}^{\mathtt{b}}_{\mathrm{G},g,p} & \mathrm{gdh} \in U^i_t, \\ \mathrm{Adv}_{\mathrm{psk}} & \mathrm{psk} \in U^i_t \end{Bmatrix},$$

and

$$\mathrm{Adv}^{\mathrm{dec}}_{\mathrm{ch,ind}} := q^{\mathrm{dec}}_{\mathrm{ch}} \min \begin{Bmatrix} \mathrm{Adv}^{\mathtt{b}}_{\mathrm{G},g,p}, & \mathrm{gdh} \in U^i_t, \\ \mathrm{Adv}_{\mathrm{psk}}, & \mathrm{psk} \in U^i_t \end{Bmatrix},$$

Similarly, for ANON security, for each non-empty set of uncorrupted keys $U^i_t \subseteq S^i_t$, with advantages as defined in Definition 15, it holds that

$$\mathrm{Adv}^{\mathrm{enc}}_{\mathrm{ch,anon}} := q^{\mathrm{enc}}_{\mathrm{ch}} \min \begin{Bmatrix} \mathrm{Adv}^{\mathtt{b}}_{\mathrm{G},g,p} & \mathrm{gdh} \in U^i_t, \\ \mathrm{Adv}_{\mathrm{psk}} & \mathrm{psk} \in U^i_t \end{Bmatrix}.$$

By summing up the above terms as $\mathrm{Adv}^{\mathtt{pre}} + \mathrm{Adv}^{\mathrm{enc}}_{\mathrm{ch,ind}} + \mathrm{Adv}^{\mathrm{dec}}_{\mathrm{ch,ind}}$ and $\mathrm{Adv}^{\mathtt{pre}} + \mathrm{Adv}^{\mathrm{enc}}_{\mathrm{ch,anon}}$ we obtain the bounds for the IND and ANON security under a fixed set of uncorrupted keys.

Interpretation. These bounds capture both the hybrid security and the redundant security of our constructions (the latter if no primitive is itself insecure): they are secure even if all keys but one are corrupted, and they are secure even if only one of the uncorrupted keys is used with an unbroken assumption. Concretely, if the post-quantum building blocks turn out to be insecure, the classic assumptions provide security as long as the classic keys remain uncorrupted, and the post-quantum assumptions provide security against quantum adversaries as long as the respective keys remain uncorrupted.

Appendix K. Proofs for Sealed PQXDH SAKE

K.1. KIND (Theorem 2)

Proof. We prove the theorem via a game-hopping proof. We formally assume that signature key sk and identity key $\mathrm{IK}^{\mathrm{sk}}$ are independently generated in this and our other SAKE proofs; we sketch in Appendix M how this can be overcome. We first prove security assuming $\mathrm{Trivial} = \mathrm{Ind}_{\mathrm{bl}}$; we then describe the relevant simplifications that can be made given $\mathrm{Trivial} = \mathrm{Ind}_{\mathrm{bl}} \wedge \mathrm{Trivial}_{\mathrm{kem}} \wedge \mathrm{Passive}$ is used.

Game 0. This is exactly game $\mathrm{KIND}^b_{\mathrm{KE,Trivial}}$ for $\mathrm{KE} = \mathrm{SealedPQXDH}$ played by adversary $\mathcal{A}$. We have

$$\mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}) = \mathrm{Adv}^{\mathrm{kind}}_{\mathrm{KE,Trivial}}(\mathcal{A}).$$

Game 1. In this game, for all MK.enc, SKE.enc, S.sgn and MAC.tag calls made by the challenger with respect to honestly generated keys, MK.dec and SKE.dec calls are replaced by the relevant input/output from MK.enc/SKE.enc, and matching S.vfy and "Require tag $=$ tag$(\mathrm{K}_{\mathrm{mac}}, \perp)$" calls are removed. Invoking the correctness of each primitive where relevant, we have, for some efficient $\mathcal{R}_1$ and $\mathcal{R}_2$ (we assume perfect correctness for SKE and MAC):

$$\left|\mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G1}}(\mathcal{A})\right| \leq \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{MK}}(\mathcal{R}_1) + \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{S}}(\mathcal{R}_2).$$

Game 2. Consider a $\mathrm{Send}(s, r, spk, pt, \mathtt{tru})$ query; let $tiid$ be its instance identifier. In this game, for all such queries for which there is no iid' such that $\mathrm{Origin}(tiid, iid')$, if the query succeeds, it must be that spk was previously output by receiver r (via a GenSPK call).

Let $\mathcal{R}$ be a SUF adversary playing with signature scheme S simulating for $\mathcal{A}$ as follows.

- GenIK: $\mathcal{R}$ calls Gen and embeds the output key in the output for $\mathcal{A}$.
- GenSPK(u): $\mathcal{R}$ uses Sgn and otherwise simulates locally.
- Send(s, r, spk, pt, ch): spk is of the form $(\mathrm{SPK}_{\mathrm{R}}, \sigma_{\mathrm{SPK}}, \mathrm{PQEK}_{\mathrm{R}}, \sigma_{\mathrm{PQEK}})$. $\mathcal{R}$ calls Vfy(r, m, σ) for $(m, \sigma) = (\mathrm{SPK}_{\mathrm{R}}, \sigma_{\mathrm{SPK}})$ and $(\mathrm{PQEK}_{\mathrm{R}}, \sigma_{\mathrm{PQEK}})$. Note that the $\mathcal{A}$ can distinguish between Game 1 and Game 2 if this query is such that $ch = \mathtt{tru}$, spk differs from spk output from GenSPK and the call succeeds, a necessary condition of the latter being that both S.vfy queries pass. By construction of spk, this implies that at least one of the Vfy calls led to $\mathcal{R}$ winning. If this doesn't occur, then the adversary simulates the rest of the call locally.
- Receive(PK, r, e, c), CorruptSSK(u, e): $\mathcal{R}$ simulates locally.
- CorruptIK(u): $\mathcal{R}$ simulates via Exp(u).

The simulation is perfect except in the case that the $\mathcal{R}$ adversary wins, and so by standard bad event analysis, we have

$$\left|\mathrm{Adv}^{\mathrm{G1}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G2}}(\mathcal{A})\right| \leq \mathrm{Adv}^{\mathrm{suf}}_{\mathrm{S}}(\mathcal{R}).$$

Game 3. Consider all keys output by the first of the two MK.enc calls KE.send (Line 07) inside of Send(..., $ch = \mathtt{tru}$) calls. Game 3 replaces each such key $\mathrm{CK}\|\mathrm{K}_{\mathrm{seal}}$ output by $\mathrm{MK}^1_{\mathrm{p}}$.enc with a uniformly random key, keeping keys with $\mathrm{MK}^1_{\mathrm{p}}$.dec outputs replaced in Game 1 consistent. In addition, for all Receive(PK, r, e, c) calls for which the corresponding instance would not be partnered *and* we have ciphertext equality,[7] $\mathcal{R}$ replaces the output of $\mathrm{MK}^1_{\mathrm{p}}$.dec with a uniformly random key.

Let $\mathcal{R}$ be an MS-KEM $\mathrm{IND}^{b'}_{\mathrm{MK},T}$ adversary simulating for some $\mathcal{A}$. $\mathcal{R}$ simulates as follows.

- GenIK: $\mathcal{R}$ simulates S.gen, calls $\mathrm{IK} \leftarrow \mathrm{Gen}(\mathtt{dh}, \perp, \perp)$ and returns the result to $\mathcal{A}$.
- GenSPK(u): $\mathcal{R}$ calls $\mathrm{SPK}_{\mathrm{R}} \leftarrow \mathrm{Gen}(\mathtt{dh}, \perp, \perp)$ and $\mathrm{PQEK}_{\mathrm{R}} \leftarrow \mathrm{Gen}(\mathtt{pqkem}, \perp, \perp)$, and otherwise simulates locally.
- Send(s, r, spk, pt, ch): $\mathcal{R}$ first simulates signature verification and calls $\mathrm{EK}_{\mathrm{S}} \leftarrow \mathrm{Gen}(\mathtt{dh}, \perp\perp)$. Then, $\mathcal{R}$ calls $\mathrm{Enc}(SPKI, \perp, \mathrm{EK}_{\mathrm{S}}, ch)$, where $SPKI$ corresponds to the relevant indices for the $\mathrm{MK}^1_{\mathrm{p}}$.enc call, which outputs $\mathrm{CK}\|\mathrm{K}_{\mathrm{seal}}$. $\mathcal{R}$ then calls Psk(CK) and then uses Enc to simulate the subsequent $\mathrm{MK}^2_{\mathrm{p}}$.enc call. $\mathcal{R}$ otherwise simulates locally.
- Receive(PK, r, e, c): If (e, c) matches an output of Send (and PK contains the sender's identity key; if it doesn't, then it returns $\perp$ immediately), then $\mathcal{R}$ simulates by correctness (due to the Game 1 hop). Otherwise, $\mathcal{R}$ simulates analogously to Send, first using Dec, then Psk and then Dec as needed.
- CorruptIK(u), CorruptSSK(u, e): $\mathcal{R}$ simulates using Corrupt.

Now, when Game 2/Game 3 is ran with bit b, $\mathcal{R}$ perfectly simulates Game 2 when its bit b' is 0 and Game 3 when $b' = 1$. It follows from the reverse triangle inequality and triangle inequality that

$$\left|\mathrm{Adv}^{\mathrm{G2}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G3}}(\mathcal{A})\right| \leq 2 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK},T}(\mathcal{R}).$$

Game 4. In this game, for the instances for which the output keys of $\mathrm{MK}^1_{\mathrm{p}}$.enc/$\mathrm{MK}^1_{\mathrm{p}}$.dec were switched to uniform in the previous hop, the corresponding $\mathrm{MK}^2_{\mathrm{p}}$.enc/$\mathrm{MK}^2_{\mathrm{p}}$.dec calls are now also switched to uniform. In particular, this means that the challenge keys output by Send are uniformly random and independent of the challenge bit, but we may still have attacks from explicit authentication at this point.

Let $\mathcal{R}$ be a MK IND adversary that simulates as follows for $\mathcal{A}$. GenIK and GenSPK queries are simulated as before. Non-challenge/changed queries are simulated as before, using Enc and Dec as needed. For challenge/changed queries, $\mathcal{R}$ simulates the first $\mathrm{MK}^1_{\mathrm{p}}$.enc/$\mathrm{MK}^1_{\mathrm{p}}$.dec calls by calling $\mathrm{Psk}(\perp)$ to simulate CK (matching consistent calls together as needed) and simulates $\mathrm{K}_{\mathrm{seal}}$ locally. $\mathrm{MK}^2_{\mathrm{p}}$.enc/$\mathrm{MK}^2_{\mathrm{p}}$.dec calls are simulated using Enc/Dec in the natural way.

By the same reasoning as the previous hop, it follows that

$$\left|\mathrm{Adv}^{\mathrm{G3}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G4}}(\mathcal{A})\right| \leq 2 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK},T}(\mathcal{R}).$$

Game 5. In this game, the line "If $\mathrm{Auth}(\mathrm{pk}, k_0, pt, iid)$: Stop with b" (line 19 of Figure 3) is removed and the game aborts if the adversary inputs a c that was not output by Send to a Receive call for instance iid that ends up succeeding and partnering with the relevant Send call. Note now that the game's execution is independent of the challenge bit, so for any $\mathcal{A}$, we have

$$\mathrm{Adv}^{\mathrm{G5}}(\mathcal{A}) = 0$$

We reduce to the security of MAC MAC. Let $\mathcal{R}$ be a SUF adversary simulating for $\mathcal{A}$ as follows. $\mathcal{R}$ simulates

7. Note we could have a case where some $c \neq c'$ is input but still forms a partnering; this is handled later in the proof.

locally except for the queries considered in the previous two hops. For these queries, $\mathcal{R}$ simulates $\mathrm{K}_{\mathrm{mac}}$ output by $\mathrm{MK}_{\mathrm{p}}^2.\mathrm{enc}/\mathrm{MK}_{\mathrm{p}}^2.\mathrm{dec}$ by calling Gen and then Tag/Vfy respectively. The simulation is perfect except if Auth succeeds or the aforementioned bad event occurs, but in these cases the MAC adversary wins, so we have

$$\left|\mathrm{Adv}^{\mathrm{G4}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G5}}(\mathcal{A})\right| \leq \mathrm{Adv}^{\mathrm{suf}}_{\mathrm{MAC}}(\mathcal{R}).$$

The result follows by collecting the probabilities.

PQ HNDL Security. We now consider security assuming $\mathrm{Trivial} = \mathrm{Ind}_{\mathrm{bl}} \wedge \mathrm{Trivial}_{\mathrm{kem}} \wedge \mathrm{Passive}$ is instead used. Since the adversary is passive, Game 2 is immediately identically distributed to Game 1, and for the same reason Game 4 and 5 are perfectly indistinguishable. Note that the signature correctness term is still in the bound, but since this is perfect for typical classical schemes like Schnorr and ECDSA, it is not an issue. Finally, since $\mathrm{Trivial}_{\mathrm{kem}}$ holds, following the logic described in Appendix J, the MS-KEM advantage terms will internally depend on a term of the form $\min\{\text{Advantage PQ-KEM}, ...\}$, and so the final bound will not be bottlenecked by any classical primitive. □

K.2. ANON (Theorem 3)

Proof. We proceed by game hopping, first assuming predicate $\mathrm{Anon}_{\mathrm{bl}}$ is used. Due to their similarity with the IND proof, we do not provide detailed reductions at all steps for our SAKE proofs hereafter.

Game 0. This is game $\mathrm{ANON}^b_{\mathrm{KE,Trivial}}$ for $\mathrm{KE} = \mathrm{SealedPQXDH}$ played by $\mathcal{A}$. We have

$$\mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}) = \mathrm{Adv}^{\mathrm{anon}}_{\mathrm{KE,Trivial}}(\mathcal{A}).$$

Game 1. In this game, for all MK.enc, SKE.enc, S.sgn and MAC.tag calls made by the challenger with respect to honestly generated keys, we replace or remove the relevant checks as in Game 1 in the IND proof. We have

$$\left|\mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G1}}(\mathcal{A})\right| \leq \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{MK}}(\mathcal{R}_1) + \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{S}}(\mathcal{R}_2).$$

Game 2. This game is defined analogously to Game 2 in the IND proof, disallowing inputs *spk* on challenge Send calls that don't correspond to a GenSPK call. We have

$$\left|\mathrm{Adv}^{\mathrm{G1}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G2}}(\mathcal{A})\right| \leq \mathrm{Adv}^{\mathrm{suf}}_{\mathrm{S}}(\mathcal{R}).$$

Game 3. This game is defined analogously to Game 3 in the IND proof, replacing the output of $\mathrm{MK}_{\mathrm{p}}^1$ encapsulation/decapsulation in TestANON and 'out-of-sync' Receive calls with uniform keys. As the $\mathrm{Anon}_{\mathrm{bl}}$ predicate is sufficiently restrictive to support MS-KEM trivial attacks, the proof strategy is the same, and so we have

$$\left|\mathrm{Adv}^{\mathrm{G2}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G3}}(\mathcal{A})\right| \leq 2 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK},T}(\mathcal{R}).$$

Game 4. This game is defined analogously to Game 4 in the IND proof (making keys output from $\mathrm{MK}_{\mathrm{p}}^2$ calls uniform) and so

$$\left|\mathrm{Adv}^{\mathrm{G3}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G4}}(\mathcal{A})\right| \leq 2 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK},T}(\mathcal{R}).$$

Game 5. This game is defined analogously to Game 5 in the IND proof: despite Auth not being enforced in the ANON game, this is safe to do since $\mathrm{Anon}_{\mathrm{bl}}$ is strictly more restrictive than $\mathrm{Ind}_{\mathrm{bl}}$, and so

$$\left|\mathrm{Adv}^{\mathrm{G4}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G5}}(\mathcal{A})\right| \leq \mathrm{Adv}^{\mathrm{suf}}_{\mathrm{MAC}}(\mathcal{R}).$$

Game 6. This game replaces the corresponding symmetric ciphertexts with uniformly random values. We reduce to the IND security of SKE. Note that our previous hops allow us to use a SKE IND definition that does not capture CCA attacks. By applying and combining two reductions, one challenging plaintexts pt_0 and the other pt_1, we have

$$\left|\mathrm{Adv}^{\mathrm{G5}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G6}}(\mathcal{A})\right| \leq 2 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{SKE}}(\mathcal{R}).$$

Game 6 simulation. We reduce directly to MS-KEM ANON security. Note that we only need to invoke ANON security once at this point, since $\mathrm{MK}_{\mathrm{p}}^2$ does not output a ciphertext.

In more detail, let $\mathcal{R}$ be an ANON adversary simulating for $\mathcal{A}$ as follows. $\mathcal{R}$ simulates GenIK and GenSPK via Gen, and otherwise simulates locally. $\mathcal{R}$ simulates TestANON calls by calling Enc with respect to the corresponding keying material of the two senders s_0 and s_1 in particular to simulate $\mathrm{ct}_{\mathrm{MK}}$. Send and Receive are then simulated using Enc, Dec and Psk naturally, and analogously for CorruptIK and CorruptSSK via Corrupt.

We then arrive at

$$\mathrm{Adv}^{\mathrm{G6}}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{anon}}_{\mathrm{MK},T}(\mathcal{R})$$

completing the proof.

PQ HNDL Security. For the same reasons as in the KIND proof, with $\mathrm{Trivial} = \mathrm{Anon}_{\mathrm{bl}} \wedge \mathrm{Trivial}_{\mathrm{kem}} \wedge \mathrm{Passive}$ the signature security terms will disappear from the bound and the MS-KEM bounds will internally not depend on Diffie-Hellman assumptions. □

Appendix L. Proofs for Sealed RingXKEM-XHMQV SAKE

L.1. KIND (Theorem 4)

Proof. We proceed by game hopping.

Game 0. This is game $\mathrm{KIND}^b_{\mathrm{KE,Trivial}}$ for $\mathrm{KE} = \mathrm{SealedRingXKEM\text{-}XHMQV}$ played by $\mathcal{A}$. We have

$$\mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}) = \mathrm{Adv}^{\mathrm{kind}}_{\mathrm{KE,Trivial}}(\mathcal{A}).$$

Game 1. All in-sync primitive queries are replaced by invoking their relevant correctness property; we have

$$\left|\mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G1}}(\mathcal{A})\right| \leq \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{MK}}(\mathcal{R}_1) + \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{S}}(\mathcal{R}_2) + \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{RS}}(\mathcal{R}_3)$$

Game 2. This game is defined analogously to Game 2 in the SealedPQXDH proofs (disallowing non-trivial signature forgeries), except for a given spk, we disallow corresponding inputs to challenge Send calls unless they match on $(\mathrm{SPK}_{\mathrm{R}}, \sigma_{\mathrm{SPK}}, \mathrm{PQEK}_{\mathrm{R}})$, corresponding to F_{origin}. Note that because this doesn't include the ring signature, we do not need strong unforgeability. One can reduce to the security of both the signature scheme and the ring signature scheme, and so we obtain

$$\left|\mathrm{Adv}^{\mathrm{G1}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G2}}(\mathcal{A})\right| \leq \min\{\mathrm{Adv}^{\mathrm{suf}}_{\mathrm{S}}(\mathcal{R}), \mathrm{Adv}^{\mathrm{unf}}_{\mathrm{RS}}(\mathcal{R})\}.$$

Game 3. Analogously to Game 3 in the SealedPQXDH proofs, in this game, the keys output by $\mathrm{MK}^1_{\mathrm{h}}.\mathrm{enc}/\mathrm{MK}^1_{\mathrm{h}}.\mathrm{dec}$ calls inside challenge Send and 'out-of-sync' Receive calls respectively are replaced by uniform keys. We have

$$\left|\mathrm{Adv}^{\mathrm{G2}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G3}}(\mathcal{A})\right| \leq 2 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK},T}(\mathcal{R}).$$

Game 4. Like Game 5 in the SealedPQXDH KIND proof, in Game 4 here the line "If $\mathrm{Auth}(\mathrm{pk}, k_0, pt, iid)$: Stop with b" (line 19 of Figure 3) is removed and the game aborts if the adversary inputs a c that was not output by Send to a Receive call for instance iid that ends up succeeding and partnering with the relevant Send call. We can reduce to 1. ring signature; and 2. a combination of MS-KEM and MAC security here. Intuitively, Diffie-Hellman provides implicit authentication (and then the MAC makes it explicit) and ring signatures provide explicit authentication.

The reduction to ring signature security is straightforward. The second proof proceeds as follows:

- First, we reduce to MS-KEM security by replacing $\mathrm{MK}^2_{\mathrm{h}}.\mathrm{enc}/\mathrm{MK}^2_{\mathrm{h}}.\mathrm{dec}$ calls with uniform keys.
- Now that the MAC key is uniform, we disallow injections. Note here that we MAC the ring signature σ since we only assume EUF RS security, and so any such malformation of σ will result in the MAC check failing (unless a MAC forgery is made).
- We then reduce to MS-KEM security to make the keys 'real' again.

We arrive at

$$\left|\mathrm{Adv}^{\mathrm{G3}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G4}}(\mathcal{A})\right| \leq \min\{\mathrm{Adv}^{\mathrm{unf}}_{\mathrm{RS}}(\mathcal{R}), 4 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK},T}(\mathcal{R}) + \mathrm{Adv}^{\mathrm{suf}}_{\mathrm{MAC}}(\mathcal{R})\}$$

Game 5. In this game, we replace the output of relevant $\mathrm{MK}^2_{\mathrm{h}}.\mathrm{enc}/\mathrm{MK}^2_{\mathrm{h}}.\mathrm{dec}$ calls with uniform keys. We have

$$\left|\mathrm{Adv}^{\mathrm{G4}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G5}}(\mathcal{A})\right| \leq 2 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK},T}(\mathcal{R}).$$

Finally, observe that $\mathrm{Adv}^{\mathrm{G5}}(\mathcal{A}) = 0$.

Hybrid Security. Our bound captures hybrid security. To see this, we consider how to interpret the bound depending on whether PQ or classical cryptography is assumed to be secure:

- Classic cryptography is broken: Ignore the $\mathrm{Adv}^{\mathrm{suf}}_{\mathrm{S}}(\mathcal{R})$ and $4 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK},T}(\mathcal{R}) + \mathrm{Adv}^{\mathrm{suf}}_{\mathrm{MAC}}(\mathcal{R})$ terms in the security bound; the second term only makes sense if the underlying MS-KEM uses 'both'-type post-quantum keys like Diffie-Hellman, which is not the case since the construction only uses PQ-KEM.
- PQ cryptography is broken: Ignore the $\mathrm{Adv}^{\mathrm{unf}}_{\mathrm{RS}}(\mathcal{R})$ terms in the bound and rely on the 'full' classical security of the MS-KEM.

□

L.2. ANON (Theorem 3)

Proof. We proceed by game hopping.

Game 0. This is game $\mathrm{ANON}^b_{\mathrm{KE,Trivial}}$ for $\mathrm{KE} = \mathrm{SealedRingXKEM\text{-}XHMQV}$ played by $\mathcal{A}$. We have

$$\mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}) = \mathrm{Adv}^{\mathrm{anon}}_{\mathrm{KE,Trivial}}(\mathcal{A}).$$

Game 1. This is defined by applying transformations from the IND proof from games 1 to 5. It follows that

$$\begin{aligned}\left|\mathrm{Adv}^{\mathrm{G0}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G1}}(\mathcal{A})\right| &\leq \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{MK_h}}(\mathcal{R}) + \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{S}}(\mathcal{R}) \\ &+ \mathrm{Adv}^{\mathrm{corr}}_{\mathrm{RS}}(\mathcal{R}) + 2 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK_h},T}(\mathcal{R}) \\ &+ \min\{\mathrm{Adv}^{\mathrm{suf}}_{\mathrm{S}}(\mathcal{R}), \mathrm{Adv}^{\mathrm{unf}}_{\mathrm{RS}}(\mathcal{R})\} \\ &+ \min\{\mathrm{Adv}^{\mathrm{unf}}_{\mathrm{RS}}(\mathcal{R}), 4 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{MK_h},T}(\mathcal{R}) + \mathrm{Adv}^{\mathrm{suf}}_{\mathrm{MAC}}(\mathcal{R})\}.\end{aligned}$$

Game 2. We reduce to SKE security in analogy with Game 6 of the proof of SealedPQXDH ANON security (Appendix K.2). We have

$$\left|\mathrm{Adv}^{\mathrm{G1}}(\mathcal{A}) - \mathrm{Adv}^{\mathrm{G2}}(\mathcal{A})\right| \leq 2 \cdot \mathrm{Adv}^{\mathrm{ind}}_{\mathrm{SKE}}(\mathcal{R}).$$

Game 2 simulation. Finally, we reduce to ANON security of MK in analogy with the proof of SealedPQXDH security, and so we have

$$\mathrm{Adv}^{\mathrm{G2}}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{anon}}_{\mathrm{MK},T}(\mathcal{R}).$$

Hybrid Security. The same argument from the previous subsection holds. □

Appendix M. Modelling Key Reuse

When proving the security of our two SAKE protocols (Figures 5 and 6), we assumed that the Diffie-Hellman key pair $(\mathrm{IK}^{\mathrm{sk}}, \mathrm{IK})$ and the (classically-secure) signature key pair $(\mathrm{sk}, \mathrm{vk})$ are independently generated. However, following protocols in the X3DH family, these keys will actually be the same when deployed in practice. The analysis

of the XHMQV protocol in [21] formally addresses this key reuse issue. We argue here that a comparable strategy can be applied for our two protocols.

In order to prove security under key reuse, the authors of [21] introduce a security notion they dub one-message unforgeability with a DDH oracle, or EUF-opCMA-DDH. Here, security requires that the signature is only used once to sign a given message, which is sufficient to prove XHMQV secure in their model (which can generally be considered to be true for X3DH-like protocols), if the underlying signature scheme is (EC)DSA or Schnorr.[8]

Like our proofs, their proof of XHMQV's security has a hop in which they exclude non-trivial signature forgeries from the identity key (i.e., prekey bundles).[9] Note that this key is also used as part of the key exchange, in their case (and notation) for deriving a term of the form $(B^{e_1}S)^{x+da} = (XA^d)^{e_1 b+s}$ (ignoring one-time prekeys) for Alice's identity key (a, g^a) and Bob's (b, g^b). In order to correctly simulate this without knowledge of say a, they model the KDF that computes the output session key as a random oracle. Then, the reduction can program the RO and check using the DDH oracle whenever it is called whether the resulting input $(B^{e_1}S)^{x+da}$ is a valid DDH tuple based on additional context fed as input to the RO.

By contrast, the fact we use MS-KEM draws an additional abstraction boundary, which prevents such a direct proof strategy. Nonetheless, the exact same DDH-oracle-based approach could be applied for our two key exchange protocols (note we already assume gap-DH and the ROM). Our protocols make two KDF calls, and all such calls are within the MS-KEMs (Lines 18 and 22 and Lines 28 and 32 of Figure 2, respectively, for encapsulation). By design, these KDF calls take input context, and so for each of our protocols (assuming a more black-box proof strategy), the reduction could analogously program the KDF and use the DDH oracle and these input context terms to check for consistency for terms involving identity keys $\mathrm{IK_S}$/$\mathrm{IK_R}$.

At this point, the XHMQV proof then reduces to the security of the identity key when used for key exchange. In order to make this work, however, the authors require at certain points that signatures can be simulated using only public keys, since the reduction may not have the relevant secret keys when it would otherwise need to. To this end, they leverage the so-called δ-simulatability of the underlying signature scheme [58], which they argue holds for (EC)DSA and Schnorr.

Again, while we cannot directly make such an argument for our protocols because MS-KEM abstracts away the KDF, it can nonetheless be seen that the same approach can be made to work.

Thus, we believe that our protocols can be formally proven secure under key reuse, and we leave this as meaningful future work to do so.

8. This is a single-user security notion, and so we could similarly use such a notion and prove security non-tightly, or introduce a multi-user notion with exposures in analogy to our current signature security notion (Figure 19).

9. Before this, the authors hop to exclude collisions on honestly-generated Diffie-Hellman keys; our proofs could introduce the same and otherwise go through.